



Course Modification: CIT 85 - HTML and CSS

Subject CIT

Discipline(s):

Computer Information Systems (Computer network installation, microcomputer technology, computer applications)

Formerly:

Cross Listed Courses:

Term Effective: 2027 Spring Semester

Course Control Number (CB00): CCC000443995

CB03: Top Code: 0707.10 - Computer Programming*

CB09: Sam Code: D - Possibly Occupational

CC Approval: 10/17/2022

Board of Trustees: 12/13/2022

Units/Hours

Total Units: 3.00

Lecture Units: 3.00

Lab Units: 0.00

Activity Units: 0.00

Number of Weeks 18

Lecture Hours Per Week: 3.00

Lab Hours Per Week: 1.00

Activity Hours: 0.00

Total In Class Contact Hours: 72.00

Total Out of Class Contact Hours: 108.00

Total Student Learning Hours: 180.00

Number of Repeats for Credit 0

Grading Method Letter Grade

Open Entry/Open Exit No

Requisites

Prerequisite: CIT 82 CIT 15

Corequisite: None

Advisory: None

Anti Requisite: None

Course Description

This course teaches students how to design web pages using HTML and CSS. Students will also gain an introduction to working with a code editor, and gain insight into creating software. (A, CSU)

Transfer/General Education

Requested Credit Classification: D - Credit - Degree Applicable

Transfer Status (CB05): B - Transferable to CSU only.

CSU Transfer Course

- CSU Transfer

CSU Transfer Course

- CSU Transfer Course

C-ID:

Objectives

1. Publish websites to a web server using industry-standard file management and deployment practices.
2. Create semantic, structured web pages using modern HTML5 standards.
3. Apply CSS to control typography, layout color systems, and visual media across a web document.
4. Construct responsive web layouts by authoring CSS media queries and utilizing fluid layout modules (Flexbox and CSS Grid) to adapt interfaces across diverse device breakpoints.
5. Integrate and optimize visual media assets for enhanced web performance and accessibility compliance.
6. Utilize AI tools to explain, troubleshoot, and debug manual HTML and CSS code syntax.
7. Audit web pages for W3C web standards compliance and WCAG accessibility,

Methods of Instruction

Multiple Methods of Instruction that may be used to provide multiple means of representation of course content in alignment with Universal Design for Learning principles.

- Lecture/Discussion
- Laboratory

Other Method

1. Lecture and Guided Interactive Demonstrations
2. Active Laboratory and Experiential Learning
3. Collaborative Technical Review & Discussion
4. Case Studies & Real-World Asset Analysis

Assignments

Multiple representative assignments that may be used to provide multiple means of engagement and expression in alignment with Universal Design for Learning principles.

Required Reading

Industry Documentation: Selected modules from the MDN Web Docs (Mozilla Developer Network) on semantic HTML5 elements, the CSS Box Model, and layout systems (Flexbox and CSS Grid).

Accessibility Standards: Comprehensive overviews of the W3C Web Accessibility Initiative (WAI) and WCAG (Web Content Accessibility Guidelines) checklist requirements for images, forms, and contrast ratios.

Required Writing

Weekly Technical Reflection Posts: Students maintain a weekly digital portfolio log explaining core CSS architectural decisions (e.g., why a specific Flexbox configuration or relative unit like rem was chosen for a layout).

AI Refactoring Documentation: A structured written assignment where students document a manual code snippet that wasn't working, the specific technical prompt they gave to an AI debugging assistant, the AI's explanation of the bug, and how the student manually implemented the corrected syntax.

Examples of Out of Class Reading and Writing

1. Structural Wireframing to Semantic Layout

Assignment: Given a physical design mock-up or a text-based layout scenario, students will sketch a structural wireframe mapping out the HTML5 document hierarchy. They will then translate that sketch into a hand-coded, syntactically correct HTML5 document using proper structural tags (

,
,
,
,
,
).

Critical Thinking Component: Students analyze the content hierarchy to ensure a logical reading flow for screen readers and AI web scrapers.

2. The Responsive Breakpoint Blueprint

Assignment: Students build a multi-column landing page that automatically adapts to mobile, tablet, and desktop viewports. Students must hand-code the macro-layout shifts utilizing CSS media queries and relative units (em, rem, vw).

Other

Critical Thinking:

Creating a semantic web document requires that students analyze content hierarchies and design problems to engineer a logical structural and visual solution. That solution must be translated into precise, standards-compliant source code using HTML5 structures and CSS layout algorithms (such as Flexbox, Grid, and media queries). Deriving a solution is a highly creative and iterative process that requires considerable critical thought. Students must actively evaluate code behavior across varied device viewports and critically analyze AI-assisted feedback to independently diagnose syntax errors, layout reflow conflicts, and web accessibility (WCAG) barriers, ensuring the final product adheres to professional web standards.

Methods of Evaluation

Multiple Methods of Evaluation that may be used to provide multiple means of engagement and expression in alignment with Universal Design for Learning principles.

Written Assignments

Substantial writing assignments are required (25% or more of grade).

Other written Assignments

The AI Refactoring Log: A written technical journal where students document a specific layout or syntax error they encountered, the exact prompt they crafted to troubleshoot it within a local LLM environment, an analysis of the AI's explanation, and a reflection on how they manually fixed the code.

Noncomputational

Structural Content Mapping: Analyzing a piece of arbitrary, unstructured content (such as a raw news article text) and designing a logical, hierarchical wireframe layout on paper or a markdown file using appropriate semantic HTML5 containers.

Other Skill Demonstration

Semantic HTML/CSS Hand-Coding: Authoring syntactically valid, structured web files from scratch using a local code editor (like VS Code), ensuring complete compliance with modern W3C specifications. Responsive Layout Construction: Utilizing Flexbox and CSS Grid to engineer fluid components that dynamically scale across varying simulated device interfaces using browser developer tools. AI-Assisted Scaffolding & Auditing: Demonstrating the ability to construct complex web elements (like multi-input accessible forms) by generating semantic baseline scaffolding using highly structured AI prompting techniques, followed by manual styling. Deployment Operations: Successfully executing industry-standard file transfer operations to deploy and publish completed local web projects to a remote web server.

Other Assessment

Interactive code reviews and peer evaluations. Skill-based quizzes testing knowledge of specific CSS properties, selector naming conventions, and technical prompt engineering strategies. A comprehensive, multi-page final website project deployed live to the internet, serving as a cumulative portfolio asset.

Texts, Other Readings, and Materials:

Other:

Open Educational Resources. All materials will be provided to students by the instructor.

Course Content

Lecture Content:

I. Web Architecture & Semantic Foundations

- **A.** Introduction to the client-server model and the mechanics of HTTP/HTTPS requests.
- **B.** Internet file management: Directory structures, root folders, and absolute vs. relative file paths.
- **C.** Evolution of markup standards: Transitioning from legacy structures to modern HTML5.
- **D.** Document object hierarchy: The global anatomy of an HTML document (`<!DOCTYPE>`, `<html>`, `<head>`, `<body>`).

- **E.** Semantic text architecture: Sectioning elements (<header>, <nav>, <main>, <section>, <article>, <footer>) vs. non-semantic block/inline containers (<div>,).

II. The Mechanics of the CSS Cascade & Presentation

- **A.** The role of Cascading Style Sheets (CSS) in separating presentation from structure.
- **B.** Syntax rules: Rulesets, selectors, declarations, properties, and values.
- **C.** Selector anatomy: Element, class, ID, attribute selectors, and pseudo-classes.
- **D.** The Cascade engine: Understanding specificity calculations, rule inheritance, and the !important flag.
- **E.** Visual properties: Managing typography, custom web fonts, hexadecimal and RGB/RGBA color spaces, and background media assets.

III. The CSS Box Model & Component Physics

- **A.** Core Box Model anatomy: Content dimensions, padding, borders, and margins.
- **B.** Box-sizing constraints: content-box vs. border-box calculation methodologies.
- **C.** Element display modes: Block, inline, inline-block, and complete layout suppression via none.
- **D.** The visible flow: Standard document flow, relative positioning, absolute positioning, and fixed elements.
- **E. AI-Skilled Workflow:** Using AI as a 24/7 styling tutor; submitting layout bugs to an LLM to diagnose accidental margin-collapse or container overflow behaviors.

IV. Modern Fluid Layout Modules

- **A.** Layout evolution: Overcoming legacy constraints (the elimination of frames and tables for layout design).
- **B.** Flexbox (Flexible Box Layout): One-dimensional layouts, axis alignment, distribution physics, and flex growth/shrink rules.
- **C.** CSS Grid Layout: Two-dimensional grid systems, explicit vs. implicit tracks, grid areas, and template alignments.
- **D. AI-Skilled Workflow:** Prompting AI to translate complex, multi-column wireframes into logical Grid structural declarations.

V. Responsive Web Design (RWD) & Mobile-First Strategy

- **A.** Principles of responsive architecture and the critical role of the meta viewport tag.
- **B.** Sizing mechanics: Absolute units (px) vs. modern relative responsive units (em, rem, vw, vh, %).
- **C.** Conditional styling: Designing macro-layout shifts utilizing CSS media queries and device breakpoints.
- **D.** Asset optimization: Fluid images, media scaling, and optimizing for varied data bandwidth conditions.

VI. Forms, Interactivity, & System Scaffolding

- **A.** Form structure: The <form> container, action attributes, and data methods.
- **B.** Interface components: Text inputs, textareas, checkboxes, radio buttons, select dropdowns, and submission triggers.
- **C.** User experience styling: Focus states, hover behaviors, and placeholder management.

VII. Quality Control: Accessibility, Validation, & AI-Readability

- **A.** Global code standards: W3C validation rules and debugging syntax errors.

- **B.** Digital inclusion: Web Content Accessibility Guidelines (WCAG) core principles, semantic reading order, image alternative text (alt), and ARIA basics.
- **C.** Search engine optimization (SEO): High-value metadata, semantic hierarchy, and configuring layouts for AI web scrapers.

Lab Content:

I. Laboratory Operations & Deployment Exercises

- **A.** Installing and customizing a local integrated development environment (IDE) like VS Code.
- **B.** Constructing standard project directory trees locally.
- **C.** Executing file transfers to deploy and publish completed coding projects live to a remote web server.

II. Hand-Coding Structural & Presentation Projects

- **A.** Writing raw, valid HTML5 text documents using structured header, main, and article blocks.
- **B.** Externalizing styles: Linking HTML documents to independent CSS files.
- **C.** Writing custom CSS rulesets to format typography, colors, and backgrounds.

III. Box Model & Layout Simulations

- **A.** Building component items to calculate padding, borders, and margins manually using browser inspector tools.
- **B.** Creating multi-column web page components utilizing Flexbox alignment properties.
- **C.** Engineering complete macro-layouts from sketch wireframes utilizing CSS Grid template areas.

IV. Responsive Engineering & Media Query Implementation

- **A.** Authoring manual media queries to transition a layout seamlessly from mobile to desktop screens.
- **B.** Implementing modern rem and percentage styling rules to ensure scalable, fluid designs.

V. Interactive Form Design & Scaffolding

- **A.** Building user registration and data submission forms with accessible labeling constraints.
- **B.** Using local LLMs to generate extensive, valid form element chunks, then manually styling their layout and interactions.

VI. Debugging, Auditing, & Refactoring Workstations

- **A.** Using the W3C markup validator to clean up layout files and fix nesting mistakes.
- **B. AI Debugging Practicum:** Submitting broken layout files to a local AI environment to pinpoint syntax or specificity errors, then manually refactoring the files based on the explanation.
- **C.** Generating technical markdown documentation tracking the improvement steps taken from original code draft to final deployment.

Student Learning Outcomes

1. Design, hand-code, and publish a multi-page responsive website using modern semantic HTML5 structures and CSS layout techniques (including Flexbox, Grid, and media queries) that conforms to current industry deployment standards.
2. Utilize AI-driven development tools as an assistant to troubleshoot syntax errors, explain layout logic, and audit source code for W3C standards and web accessibility compliance.

Diversity, Equity, and Inclusion

Description of approaches faculty may use to accommodate and engage diverse student bodies, advance equitable student outcomes, and promote the inclusion of all students (Title 5 § 55001).

Description of Approaches

Faculty promote diversity, equity, and inclusion in CIT 85 through student-centered instructional practices that support learners with varied technical backgrounds, preparation levels, and educational goals. Course materials and evaluations are structured to eliminate historical barriers to technology entry through the deployment of early diagnostic success audits and supportive bridge labs. By integrating structured AI prompting for code comprehension, debugging, and syntax explanation, the curriculum democratizes access to front-end web authoring, allowing students without prior technical exposure to successfully analyze, organize, and hand-code semantic HTML and CSS. Course projects are consistently connected to diverse, real-world professional contexts—including web accessibility (W3C/WCAG) compliance to foster inclusion for users with disabilities—and zero-cost digital resources ensure financial barriers do not impede student equity, fostering an inclusive environment for all learners entering the modern digital workforce.